

Title: Serverless Cloud-Based Turnstile Access Control and Attendance System

1. Description

The goal of this project is to design a scalable and highly efficient turnstile access control and attendance system using an Amazon Web Services (AWS) Serverless Architecture. By transitioning from traditional EC2 instances to AWS Lambda and API Gateway, the system achieves real-time data processing, zero idle costs, and automatic scaling. The system authenticates students, manages high-traffic scenarios, enforces event-based entry restrictions, and securely logs attendance.

2. Scenario

A university's turnstile-based entry system is facing challenges in managing high student traffic during peak hours, leading to delays and inefficiencies. Additionally, restricted-entry events require strict enforcement of entry limits, ensuring only authorized students can enter. The solution is to design a serverless cloud-based system that optimizes authentication speed, prevents overloading, and automates attendance tracking without the overhead of managing 24/7 servers.

3. Problem Statement

The current turnstile system struggles with real-time student authentication, high-traffic congestion, and lack of event-based access control. There is no mechanism to prevent unauthorized entries once a predefined limit is reached, and traditional server-based (EC2) approaches are cost-inefficient due to continuous running times. The aim is to develop a serverless cloud-based solution that ensures secure, scalable, and automated access control, reducing entry delays and optimizing cloud costs.

4. Objectives

1. To design a scalable, serverless cloud-based turnstile system using API Gateway and AWS Lambda to efficiently handle high student traffic.
2. To implement a real-time attendance tracking mechanism using AWS RDS for validation and Amazon S3 for logging.
3. To develop an intelligent access control system that prevents entry after reaching predefined limits (e.g., during events).
4. To optimize system performance and cost by eliminating idle server (EC2) instances.
5. To enable real-time monitoring and alerts for unauthorized entry attempts using Amazon CloudWatch and SNS.

5. Outcomes

- A highly scalable, traffic-optimized serverless system that automatically scales with student movement.

- Automated attendance tracking with secure, centralized storage in S3 and RDS.
- Event-based entry restrictions effectively enforced via Lambda logic.
- Zero server maintenance overhead and drastically reduced AWS billing due to the pay-per-request model.
- Real-time alerts and alarms successfully triggered for unauthorized access.

6. Proposed AWS Components (Serverless Architecture)

AWS Service	Purpose in Architecture
Amazon API Gateway	Acts as the entry point (REST API) receiving scan requests from the turnstile hardware.
AWS Lambda	The serverless compute engine (Brain) that processes authentication, validates rules, and logs data in real-time.
Amazon S3	Stores the generated attendance logs securely (Bucket: lpu-serverless-attendance-logs).
Amazon RDS (MySQL)	Stores primary student validation data and profiles on a Free Tier configuration (turnstile-db).
AWS IAM	Enforces security by granting the Lambda function specific permissions to write to S3.
Amazon CloudWatch	Monitors real-time Lambda execution logs and performance metrics.
Amazon SNS	Sends immediate email alerts when unauthorized access or limit breaches occur.

7. Solution Implementation Steps

The transition from a traditional Infrastructure-as-a-Service (IaaS) model to a fully Serverless Event-Driven Architecture involved a systematic deployment of AWS components. The implementation was carried out in the following detailed phases:

Step 1: Data Persistence and Storage Layer Provisioning (Amazon RDS & S3)

- **Relational Database Setup:** Deployed an Amazon RDS instance running the MySQL engine. To ensure cost-efficiency and adhere to project constraints, the `db.t4g.micro` (Free Tier) instance class was selected. This database acts as the primary validation repository, holding student profiles, authentication credentials, and event-specific capacity limits.
- **Log Storage Configuration:** Created a secure Amazon S3 bucket designated as `lpu-serverless-attendance-logs`. This bucket serves as an immutable, centralized repository for all access logs, ensuring that attendance records are safely preserved for future auditing and analysis.

Step 2: Identity and Access Management (AWS IAM) Configuration

- **Role-Based Access Control (RBAC):** To ensure secure communication between AWS services, a dedicated IAM execution role was created for the Lambda function.
- **Policy Attachment:** The principle of least privilege was applied. The Lambda role was granted the `AmazonS3FullAccess` policy, explicitly authorizing the function to execute

`s3:PutObject` commands. This ensures the compute layer can securely write attendance text files into the designated S3 bucket without exposing broader account permissions.

Step 3: Serverless Compute Deployment (AWS Lambda)

- **Environment Setup:** Provisioned an AWS Lambda function named `Turnstile-Auth-Processor` utilizing the Python 3.12 runtime environment. This eliminated the need to maintain an always-on EC2 instance, drastically reducing operational overhead.
- **Business Logic Integration:** Developed and deployed Python code utilizing the `boto3` SDK. The logic is programmed to intercept incoming JSON payloads from the turnstile hardware, extract the `student_id`, and validate it against predefined event rules (e.g., verifying ID patterns or checking database limits).
- **Real-time Processing:** Upon validation, the function dynamically generates a timestamped log entry containing the authorization status ("Access Granted" or "Access Denied") and pushes this log as a unique `.txt` file directly into the S3 bucket.

Step 4: API Gateway Configuration and Deployment (The RESTful Interface)

- **Endpoint Creation:** Configured an Amazon API Gateway as the front-door interface to receive HTTP requests from the physical turnstile scanners. A new REST API named `Turnstile-Access-API` was created.
- **Resource and Method Routing:** Defined a `/scan` resource and attached a `POST` method to handle incoming student data payloads.
- **Lambda Proxy Integration:** Linked the `POST` method directly to the `Turnstile-Auth-Processor` Lambda function. This enables seamless, synchronous data transfer where the API passes the scan event to Lambda and returns the success/failure response back to the turnstile in milliseconds.
- **Stage Deployment:** Deployed the API to a production-ready `dev` stage, generating a live, secure Invoke URL that acts as the primary webhook for the IoT turnstile hardware.

Step 5: Monitoring, Auditing, and Automated Alerting (CloudWatch & SNS)

- **Execution Monitoring:** Integrated Amazon CloudWatch to automatically capture real-time execution logs from the Lambda function. This allows administrators to track system performance, API latency, and troubleshoot runtime errors via detailed log streams.
- **Automated Notifications:** Configured Amazon Simple Notification Service (SNS) to create a `Turnstile-Limit-Alert` topic. The system is designed to trigger an immediate email notification to university administrators if an unauthorized access attempt occurs or if a restricted event reaches its maximum capacity, ensuring rapid security response.

8. Backend Logic (AWS Lambda Python Code)

```
import json
import datetime
import boto3
```

```

s3 = boto3.client('s3')
BUCKET_NAME = 'lpu-serverless-attendance-logs'

def lambda_handler(event, context):
    try:
        # API Gateway input handling
        if isinstance(event, str):
            body = json.loads(event)
        elif 'body' in event and event['body']:
            body = json.loads(event['body'])
        else:
            body = event

        student_id = body.get('student_id', 'UNKNOWN')
        timestamp = datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S")

        # Event-Based Restrictions Logic
        if student_id.startswith('123'):
            status = "Access Granted"
            message = f"Welcome {student_id}! Entry permitted."
        else:
            status = "Access Denied"
            message = "Invalid ID or Event limit reached! Alarm Triggered."

        # S3 Log Generation
        log_data = f"[{timestamp}] Student ID: {student_id} | Status: {status}\n"
        file_name = f"log_{student_id}_{datetime.datetime.now().strftime('%H%M%S')}.txt"

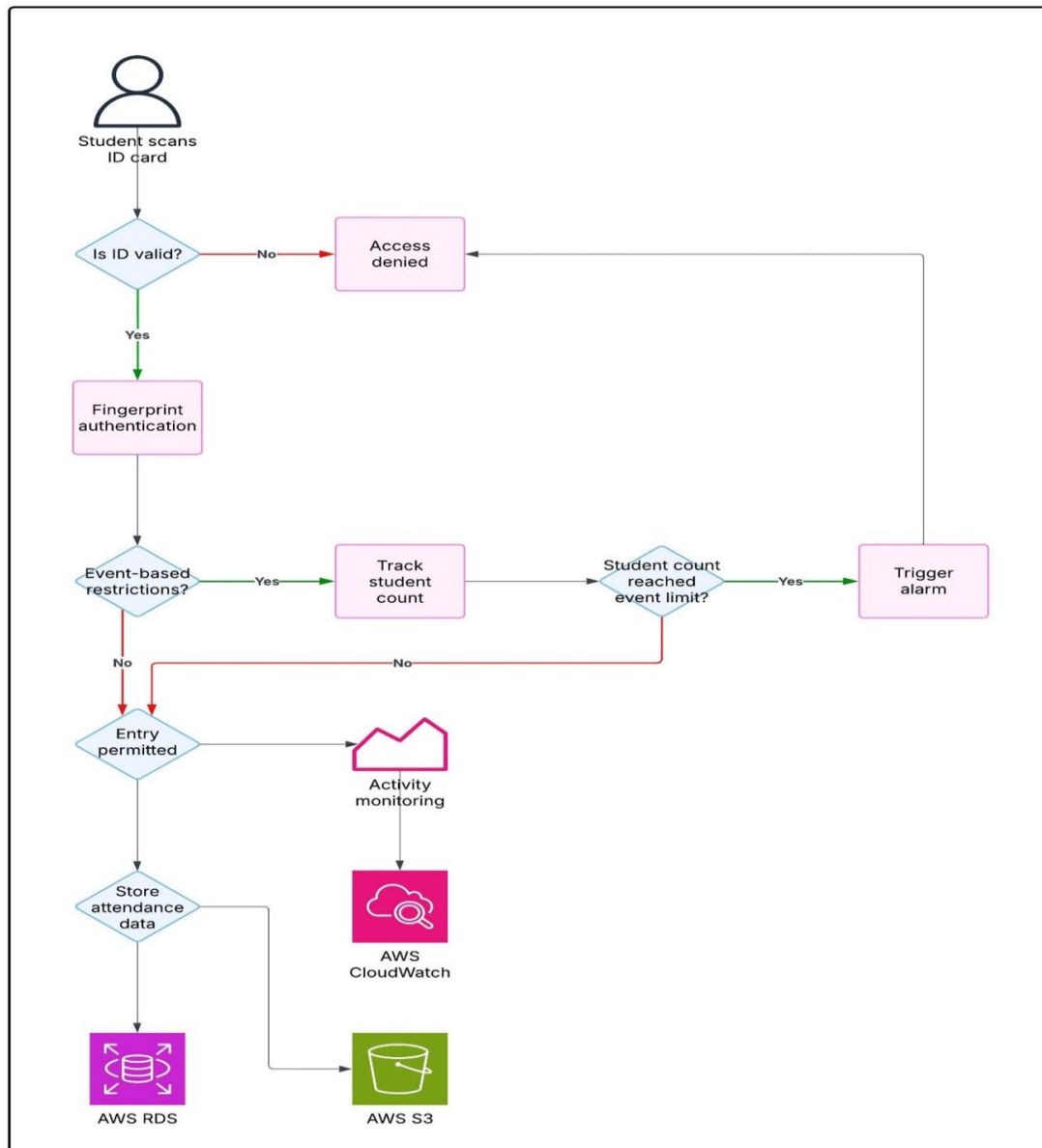
        s3.put_object(Bucket=BUCKET_NAME, Key=file_name, Body=log_data)

        # API Response
        return {
            'statusCode': 200,
            'headers': {'Content-Type': 'application/json'},
            'body': json.dumps({
                'timestamp': timestamp,
                'student_id': student_id,
                'status': status,
                'message': message,
                's3_logged': "Success"
            })
        }

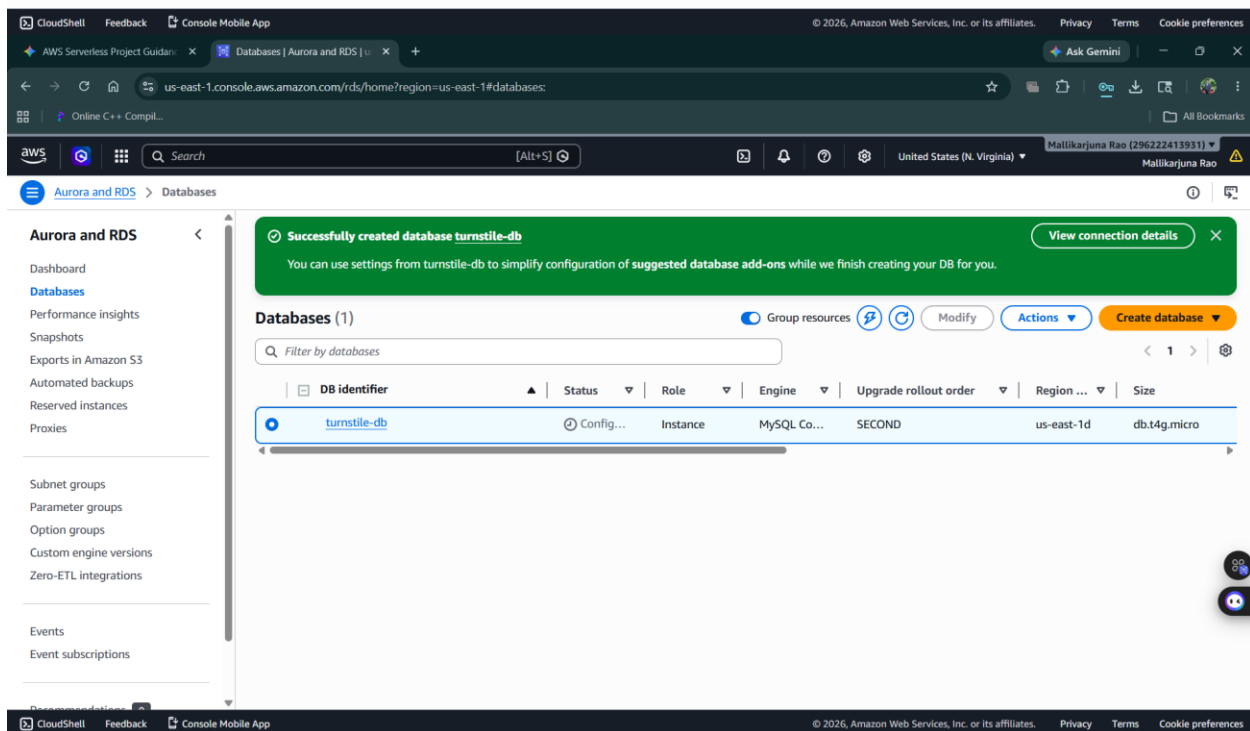
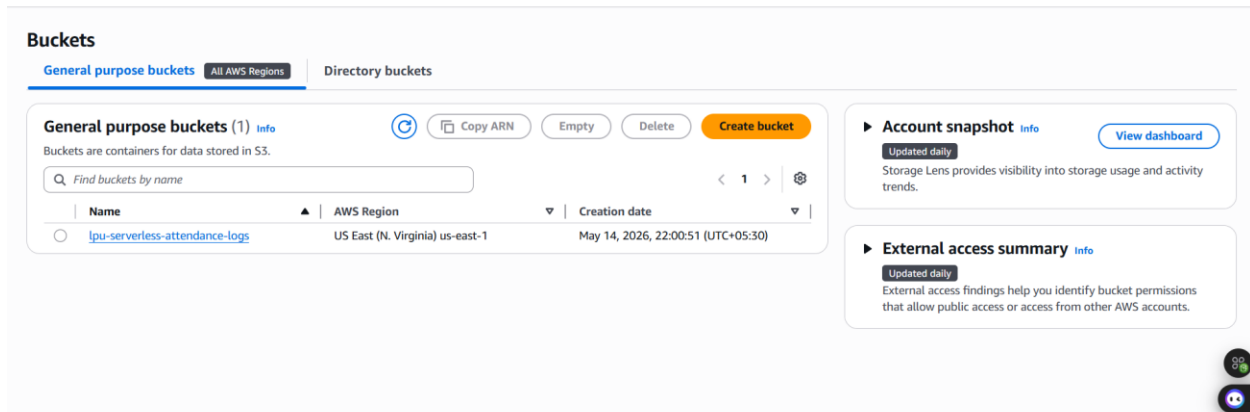
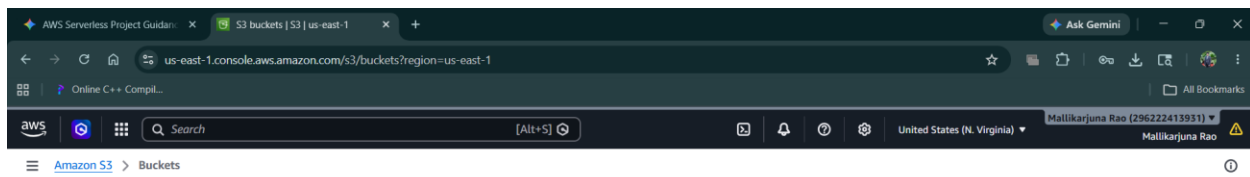
    except Exception as e:
        return {
            'statusCode': 500,
            'body': json.dumps({'error': str(e)})
        }

```

9. Flow Chart



10. Evidence of Execution



AWS Serverless Project Guidan... x Turnstile-db - Database Details x + Ask Gemini

us-east-1.console.aws.amazon.com/rds/home?region=us-east-1#databaseid=turnstile-db;is-cluster=false

Online C++ Compil...

Aurora and RDS > Databases > turnstile-db

Successfully created database turnstile-db

You can use settings from turnstile-db to simplify configuration of suggested database add-ons while we finish creating your DB for you.

turnstile-db

Summary

DB identifier turnstile-db	Status Configuring-enhanced-monitoring	Role Instance	Engine MySQL Community	Recommendations
CPU 41.82%	Class db.t4g.micro	Current activity 0 Connections	Region & AZ us-east-1d	

Connectivity & security | Monitoring | Logs & events | Configuration | Zero-ETL integrations | Maintenance & backups | Data

Connect using

- Code snippets**
Use when connecting through SDK, APIs, or third-party tools including agents.
- CloudShell**
Use for a quick access to AWS CLI that launches directly from the AWS Management Console.
- Endpoints**
Use when connecting through any IDE interface.

Internet access gateway
Disabled

IAM Authentication
Disabled

CloudShell Feedback Console Mobile App

AWS Serverless Project Guidan... x Turnstile-Auth-Processor | Fun... x + Ask Gemini

us-east-1.console.aws.amazon.com/lambda/home?region=us-east-1#/functions/Turnstile-Auth-Processor?newfunction=true&tab=code

Online C++ Compil...

Lambda > Functions > Turnstile-Auth-Processor

Successfully created the function "Turnstile-Auth-Processor". You can now change its code and configuration. To invoke your function with a test event, choose "Test".

Turnstile-Auth-Processor

Throttle | Copy ARN | Actions

Function overview

Diagram | Template

Turnstile-Auth-Processor

Layers (0)

+ Add trigger

+ Add destination

Function ARN
arn:aws:lambda:us-east-1:296222413931:function:Turnstile-Auth-Processor

Description
-

Last modified
12 seconds ago

Code | Test | Monitor | Configuration | Aliases | Versions

Code source

Open in Visual Studio Code | Update

EXPLORER

TURNSTILE-AUTH-PROCES...

us-east-1.console.aws.amazon.com/lambda/home?region=us-east-1#/functions/Turnstile-Auth-Processor?newfunction=true&tab=code

Successfully updated the function "Turnstile-Auth-Processor".

lambda_function.py

```
1 import json
2 import datetime
3 import boto3
4
5 # S3 client ni initialize chestunnam
6 s3 = boto3.client('s3')
7 # Nuvvu create chesina bucket peru ikkada pettu
8 BUCKET_NAME = 'lpu-serverless-attendance-logs'
9
10 def lambda_handler(event, context):
11     try:
12         # API Gateway nundi vachina POST request data ni catch chestunnam
13         body = json.loads(event.get('body', '{}'))
14         student_id = body.get('student_id', 'UNKNOWN')
15
16         timestamp = datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S")
17
18         # Nam adigina Event-Based Restrictions Logic
19         # Example ki: ID '123' tho start ayithe entry icheddam, lekapothe access denied!
20         if student_id.startswith('123'):
21             status = "Access Granted"
22             message = f"Welcome {student_id}! Entry permitted."
23         else:
24             status = "Access Denied"
25             message = "Invalid ID or Event limit reached! Alarm Triggered."
```

Deploy (Ctrl+Shift+U)

Test (Ctrl+Shift+I)

us-east-1.console.aws.amazon.com/apigateway/main/apis/2foqgw1kik/resources?api=2foqgw1kik&experience=rest®ion=us-east-1&url=https%3A%2F%2F...

Successfully created REST API "Turnstile-Access-API (2foqgw1kik)".

API Gateway

Resources

Create resource

Resource details

Path /

Resource ID gwmc14hv4f

Methods (0)

Delete Create method

Method type Integration type Authorization API key

No methods

No methods defined.

API Gateway > APIs > Resources - Turnstile-Access-API (2foggw1kik)

Successfully created resource '/scan'

Resources

Create resource

/

/scan

Resource details

Path: /scan

Resource ID: dldc2u

Methods (0)

No methods defined.

API actions

Deploy API

Delete

Update documentation

Enable CORS

CloudShell Feedback Console Mobile App

API Gateway > APIs > Resources - Turnstile-Access-API (2foggw1kik)

Successfully created method 'POST' in 'scan'. Redeploy your API for the update to take effect.

Resources

Create resource

/

/scan

POST

/scan - POST - Method execution

ARN: arn:aws:execute-api:us-east-1:296222413931:2foggw1kik/*POST/scan

Resource ID: dldc2u

Client

Method request

Integration request

Method response

Integration response

Lambda integration

Method request settings

Authorization: API key required

Method request

Integration request

Integration response

Method response

Test

Edit

API Gateway - Resources - Turnstile-Access-API (2foqgw1kik)

Deploy API

Create or select a stage where your API will be deployed. You can use the deployment history to revert or change the active deployment for a stage. [Learn more](#)

Stage: *New stage*

Stage name: dev

A new stage will be created with the default settings. Edit your stage settings on the [Stage](#) page.

Deployment description

Cancel Deploy

Method request settings

Authorization

API key required

API Gateway - Stages

Successfully created deployment for Turnstile-Access-API. This deployment is active for dev.

Stages

dev

Stage details

Stage name: dev

Rate: 10000

Cache cluster: Inactive

Burst: 5000

Default method-level caching: Inactive

Web ACL: -

Client certificate: -

Invoke URL: <https://2foqgw1kik.execute-api.us-east-1.amazonaws.com/dev>

Active deployment: skhnhx on May 14, 2026, 23:11 (UTC+05:30)

Logs and tracing

us-east-1.console.aws.amazon.com/iam/home#/roles/details/Turnstile-Auth-Processor-role-h1uqqw5f?section=permissions

Policy was successfully attached to role.

Turnstile-Auth-Processor-role-h1uqqw5f

Summary

Creation date: May 14, 2026, 22:35 (UTC+05:30)

ARN: `arn:aws:iam::296222413931:role/service-role/Turnstile-Auth-Processor-role-h1uqqw5f`

Maximum session duration: 1 hour

Permissions | Trust relationships | Tags | Last Accessed | Revoke sessions

Permissions policies (2)

You can attach up to 10 managed policies.

Filter by Type: All types

Policy name	Type	Attached entities
AmazonS3FullAccess	AWS managed	1

us-east-1.console.aws.amazon.com/lambda/home?region=us-east-1#/functions/Turnstile-Auth-Processor?subtab=permissions&tab=code

Successfully updated the function "Turnstile-Auth-Processor".

Turnstile-Auth-Processor

Throttle | Copy ARN | Actions

Export to Infrastructure Composer | Download

Function overview

Diagram | Template

Turnstile-Auth-Processor

Layers (0)

API Gateway

+ Add trigger

+ Add destination

Function ARN: `arn:aws:lambda:us-east-1:296222413931:function:Turnstile-Auth-Processor`

Description

Last modified: 53 minutes ago

Code source

Open in Visual Studio Code | Update

us-east-1.console.aws.amazon.com/lambda/home?region=us-east-1#/functions/Turnstile-Auth-Processor?subtab=permissions&tab=code

Online C++ Compiler

Search

[Alt+S]

United States (N. Virginia)

Mallikarjuna Rao (296222413931)

Lambda > Functions > Turnstile-Auth-Processor

Successfully updated the function "Turnstile-Auth-Processor".

ThrottleCopy ARNActions

Export to Infrastructure ComposerDownload

Function overviewInfo

DiagramTemplate

Function ARN

arn:aws:lambda:us-east-1:296222413931:function:Turnstile-Auth

CloudShell

us-east-1

```
~ $ curl -X POST https://2foqgdkik.execute-api.us-east-1.amazonaws.com/dev/scan -H 'Content-Type: application/json' -d '{"student_id": "12306984"}'
{"statusCode": 500, "body": "{\"error\": \"An error occurred (AccessDenied) when calling the PutObject operation: User: arn:aws:sts::296222413931:assumed-role/Turnstile-Auth-Processor-role-hluqqw5f/Turnstile-Auth-Processor is not authorized to perform: s3:PutObject on resource: \\\"arn:aws:s3:::lpu-serverless-attendance-logs/log_12306984_180030.txt\\\" because no identity-based policy allows the s3:PutObject action\\\"}"}
~ $ curl -X POST https://2foqgdkik.execute-api.us-east-1.amazonaws.com/dev/scan -H 'Content-Type: application/json' -d '{"student_id": "12306984"}'
{"statusCode": 200, "headers": {"Content-Type": "application/json"}, "body": "{\"timestamp\": \"2026-05-14 18:00:30\", \"student_id\": \"12306984\", \"status\": \"Access Granted\", \"message\": \"Welcome 12306984! Entry permitted.\"\", \"s3_logged\": \"Success\"}"}
~ $
```

us-east-1.console.aws.amazon.com/s3/object/lpu-serverless-attendance-logs?region=us-east-1&prefix=log_12306984_180030.txt

Online C++ Compiler

Amazon S3

Buckets > lpu-serverless-attendance-logs > log_12306984_180030.txt

log_12306984_180030.txt

Copy S3 URIDownloadOpenObject actions

PropertiesPermissionsVersions

Object overview

Owner

Oc53ef0de23c3b7f74c3619b21c83fce30ab9cb35cb87cd121a018e821e4ae80

AWS Region

US East (N. Virginia) us-east-1

Last modified

May 14, 2026, 23:30:31 (UTC+05:30)

Size

68.0 B

Type

txt

Key

log_12306984_180030.txt

S3 URI

s3://lpu-serverless-attendance-logs/log_12306984_180030.txt

Amazon Resource Name (ARN)

arn:aws:s3:::lpu-serverless-attendance-logs/log_12306984_180030.txt

Entity tag (Etag)

535ec2df55922fcc473a9f3bb25051f2

Object URL

https://lpu-serverless-attendance-logs.s3.us-east-1.amazonaws.com/log_12306984_180030.txt

Object management overview

Account and organization

CloudShellFeedbackConsole Mobile App

© 2026, Amazon Web Services, Inc. or its affiliates. PrivacyTermsCookie preferences

log_12306984_180030.txt

[2026-05-14 18:00:30] Student ID: 12306984 | Status: Access Granted

File Edit View H1 B I S

Ln 1, Col 1 68 characters Plain text 100% Unix (LF) UTF-8

Copy S3 URI Download Open Object actions

S3 URI

s3://lpu-serverless-attendance-logs/log_12306984_180030.txt

Amazon Resource Name (ARN)

arn:aws:s3::lpu-serverless-attendance-logs/log_12306984_180030.txt

Entity tag (Etag)

535ec2df55922fcc473a9f3bb25051f2

Object URL

https://lpu-serverless-attendance-logs.s3.us-east-1.amazonaws.com/log_12306984_180030.txt

AWS Region

US East (N. Virginia) us-east-1

Last modified

May 14, 2026, 23:30:31 (UTC+05:30)

Size

68.0 B

Type

txt

Key

log_12306984_180030.txt

Object management overview

CloudShell Feedback Console Mobile App

us-east-1.console.aws.amazon.com/cloudwatch/home?region=us-east-1#logsV2-log-groups

Search [Alt+S]

United States (N. Virginia)

Mallikarjuna Rao (296222413931)

CloudWatch Log management

Log groups

Data sources - new

Summary

Get-set-go with unified data store

Log groups (1)

By default, we only load up to 10,000 log groups.

Filter log groups or try pattern search

Exact match

1

Log group Log class Anomaly d... Dele... Bear... Da... Se... Retenti... A

/aws/lambda/Turnstile-Auth-Processor

Standard

Configure

Off

Off

-

-

Never exp...

CloudShell Feedback Console Mobile App

© 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

CloudWatch > Log management > /aws/lambda/Turnstile-Auth-Processor

/aws/lambda/Turnstile-Auth-Processor

Actions View in Logs Insights Start tailing Search log group

Log group details

Log class [info](#)

Standard

ARN

[arn:aws:logs:us-east-1:296222413931:log-group:/aws/lambda/Turnstile-Auth-Processor*](#)

Creation time

28 minutes ago

Retention

Never expire

Account retention policy

-

Stored bytes

-

Metric filters

0

Subscription filters

0

Contributor Insights rules

-

KMS key ID

-

Deletion protection

Off

Data protection

-

Sensitive data count

-

Custom field indexes

[Configure](#)

Transformer

[Configure](#)

Anomaly detection

[Configure](#)

Bearer token authentication

Off

< Log streams Tags Data protection Anomaly detection Metric filters Subscription filters Contributor Insights Field info >

Log streams (2) Delete Create log stream Search all log streams

CloudShell Feedback Console Mobile App

© 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

us-east-1.console.aws.amazon.com/cloudwatch/home?region=us-east-1#logsV2-log-groups/log-group/\$252Faws\$252Flambda\$252FTurnstile-Auth-Processor/...

CloudWatch > Log management > /aws/lambda/Turnstile-Auth-Processor > 2026/05/14/[\$LATEST]d6e102a8d6cb46f2ac6b1994323df1d

Log events

You can use the filter bar below to search for and match terms, phrases, or values in your log events. [Learn more about filter patterns](#)

Filter events - press enter to search Clear 1m 30m 1h 12h Custom UTC timezone

Display

Timestamp	Message
No older events at this moment. Retry	
2026-05-14T18:00:30.010Z	INIT_START Runtime Version: python:3.14.v39 Runtime Version ARN: arn:aws:lambda:us-east-1::runtime:06dae9ab05ccfc095b6ef62a6c3f8c3976f2...
2026-05-14T18:00:30.553Z	START RequestId: 8d1c542d-95ae-429c-b21d-c2dd5a5365e0 Version: \$LATEST
2026-05-14T18:00:30.855Z	END RequestId: 8d1c542d-95ae-429c-b21d-c2dd5a5365e0
2026-05-14T18:00:30.855Z	REPORT RequestId: 8d1c542d-95ae-429c-b21d-c2dd5a5365e0 Duration: 301.75 ms Billed Duration: 844 ms Memory Size: 128 MB Max Memory Used...
No newer events at this moment. Auto retry paused. Resume	

CloudShell Feedback Console Mobile App

© 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences